

PES UNIVERSITY, Bangalore

Established under Karnataka Act No. 16 of 2013)

Department of Computer Science & Engineering

UE24CS251A: DIGITAL DESIGN AND COMPUTER ORGANIZATION

Unit 4

Multiplication

Binary multiplication in digital systems is an essential arithmetic operation implemented using combinational logic. Just as decimal multiplication involves generating and summing partial products, binary multiplication follows the same concept but replaces the digit multiplications with logical **AND** operations. For two unsigned binary numbers — a multiplicand (M) and a multiplier (Q) — each bit of the multiplier multiplies every bit of the multiplicand to produce a **partial product**. Each partial product row is then shifted left according to the weight of the multiplier bit and summed to form the final binary product. When two n -bit numbers are multiplied, the resulting product has **$2n$ bits**, reflecting the increased bit-width required to represent the full result.

The hardware implementation of this process is known as an **Array Multiplier**, which organizes the computation in a systematic grid of **AND** and **Full Adder (FA)** cells. Every cell in the array performs a bit-level multiplication using an AND gate and adds the result to the corresponding partial product and carry bits using a Full Adder. The AND gate in each cell produces the product term $m_j \cdot q_i$, while the Full Adder sums it with the carry-in and previous partial product to generate the next partial product bit and a carry-out. The multiplicand bits flow vertically downward, and the multiplier bits propagate horizontally across the array. This parallel data movement makes the array multiplier a fast and regular architecture that can be easily scaled for any word length.

For an $n \times n$ array multiplier, the total number of logic elements required can be expressed as n^2 AND gates and $n(n-1)$ Full Adders. The first row of the array uses only AND gates, as there are no previous partial products to add, while each of the remaining $(n-1)$ rows performs addition operations, hence the factor $(n-1)$. As a result, for a 4×4 array multiplier, the hardware consists of 16 AND gates and 12 Full Adders. The **critical-path delay**, which determines the maximum speed of the circuit, depends on both the propagation delay of the AND gates and the Full Adders. Since the first row contributes one AND delay and subsequent rows each introduce two FA-level delays, the worst-case propagation delay can be derived as $T_{\text{delay}} = (n-1)t_{\text{FA}} + t_{\text{AND}}$, where t_{FA} and t_{AND} represent the individual delays of a Full Adder and an AND gate respectively. This expression captures the cumulative delay along the longest carry-propagation path through the array. For a 4×4 multiplier with $t_{\text{FA}}=2$ ns and $t_{\text{AND}}=1$ ns, the theoretical delay equals 7 ns, though practical implementations report an effective delay of about 23 ns due to logic overlap and gate-level optimization.

In summary, the array multiplier exemplifies an efficient, parallel combinational architecture for binary multiplication. Its regular structure simplifies hardware design and timing analysis, making it ideal for use in arithmetic logic units (ALUs) and digital signal processors (DSPs). Understanding the flow of partial products, the number of required gates, and the delay characteristics is fundamental to optimizing multiplier performance in digital systems.

1. Shift-Add Multiplication

Concept

The **Shift-Add algorithm** is a *sequential multiplication* technique for unsigned binary numbers.

It mimics paper-and-pencil multiplication but uses a single adder and sequential shifts instead of multiple parallel adders.

Registers Required

- **A (Accumulator):** Holds partial sums
- **Q (Multiplier):** Contains the multiplier bits
- **M (Multiplicand):** Number to be multiplied
- **C (Carry / Sign):** Stores carry or sign extension bit

Initially:

A = 0

Q = Multiplier

M = Multiplicand

C = 0

Algorithm Steps

1. Check the least significant bit (q_0) of Q.
 - If $q_0 = 1 \rightarrow$ Add M to A.
 - If $q_0 = 0 \rightarrow$ Do nothing.
2. **Right shift** the combined (C, A, Q) register by one bit.
3. **Repeat Steps 1 & 2** for n cycles (where n = number of bits in multiplier).
4. After n cycles, the concatenation of (A, Q) gives the **final product**.

Example: Perform multiplication of M = 1011 (11) and Q = 1101 (13)

Step	q_0	Operation	A (after add)	(C,A,Q) after shift	Comment
Init	—	—	0000	A=0000, Q=1101	—
1	1	$A \leftarrow A+M \rightarrow 0000+1011=1011$	1011	0101 1110	Add & shift

Step	q_0	Operation	A (after add)	(C,A,Q) after shift	Comment
2	0	No add	—	0010 1111	Shift only
3	1	$A \leftarrow A+M \rightarrow 0010+1011=1101$	1101	0110 1111	Add & shift
4	1	$A \leftarrow A+M \rightarrow 0110+1011=10001 \rightarrow$ (carry=1, A=0001)	0001	1000 1111	Add & shift

Final product = (A,Q) = 10001111₂ = 143₁₀.

Thus, $13 \times 11 = 143$, verified.

Characteristics

- Needs **n clock cycles** for n-bit operands.
- Efficient hardware-wise: one n-bit adder reused repeatedly.
- Used for **unsigned or positive signed** binary multiplication.
- Can be extended to signed numbers by sign extension or Booth's algorithm.

2. Booth's Algorithm

Purpose

Booth's algorithm efficiently handles **signed binary multiplication** and reduces the **number of additions/subtractions** required when the multiplier contains *blocks of consecutive 1s*. It works uniformly for **positive and negative numbers in 2's complement form**.

Key Idea

Instead of adding **M for every 1-bit in the multiplier**, Booth's algorithm **detects transitions** (from 0→1 or 1→0) in the multiplier bits to determine when to add or subtract M.

We keep an extra bit **$Q_{-1} = 0$** initially.

Decision Table

(Q_0, Q_{-1}) Operation on A Meaning

0 0	No operation	Continuous 0s
0 1	$A \leftarrow A + M$	Start of a 1-sequence
1 0	$A \leftarrow A - M$	End of a 1-sequence

(Q₀, Q₋₁) Operation on A Meaning

1	1	No operation	Continuous 1s
---	---	--------------	---------------

After each operation, **arithmetic right-shift (A, Q, Q₋₁)** by one bit.

Example Overview

Consider multiplication of $(-3) \times (+5)$:

M = 1101 (-3)

Q = 0101 (+5)

Q₋₁ = 0

Steps:

1. Check (Q₀, Q₋₁) each cycle → perform +M/-M/no-op as per table.
2. Arithmetic right-shift after every operation.
3. After 4 cycles, final (A,Q) gives **11110001 = -15**, verified result.

Advantages

- Handles **signed operands** directly (2's complement form).
- Collapses long blocks of 1s to just **two operations** (add at start, subtract at end).
- Reduces total arithmetic operations in many practical cases.

Disadvantages

- Extra logic for detecting bit transitions.
- Alternating 0-1 patterns can create more operations than simple shift-add.

3. Best- and Worst-Case Multiplier Patterns

Best Case: 01111110

Transitions: 0→1 (at bit 6), 1→0 (at bit 1)

- **A single block of 1s** exists.
- Booth interprets the whole block with **two operations**:
 - +M at the first 0→1 transition,
 - -M at the 1→0 transition.
- Intermediate 1s yield (1 1) pairs → no operation.
- Therefore, the multiplier needs **only 2 adds/subs** → fastest (best-case) scenario.

Worst Case: 010101010

Transitions alternate every bit $\rightarrow 01, 10, 01, 10, \dots$

- Each bit pair triggers an operation (+M, -M, +M, -M, ...).
- Almost every step requires addition or subtraction, leading to **maximum operations**.
- In this case, Booth's algorithm is **slower** than a simple sequential multiplier because no compression of 1s occurs.

Summary Table

Pattern	Transitions	Operations	Remarks
01111110	Two (0 $\rightarrow$ 1, 1 $\rightarrow$ 0)	+M, -M	Best case
010101010	Alternate	Many (+M, -M)	Worst case
00000000	None	No operation	Trivial
11111111	None (all 1s)	+M once	Efficient

4. Comparative Summary

Feature	Shift-Add	Booth's Algorithm
Handles signed numbers	No (extension needed)	Yes (2's complement)
Number of additions	n (for n-bit multiplier)	$\leq n/2$ (typical)
Control logic	Simple	Moderate (transition detection)
Best case speed	Same as array	Faster
Worst case speed	Moderate	Same or slower (alternate bits)

Key Takeaways

- **Shift-Add**: sequential, simpler, hardware-light.
- **Booth**: intelligent encoding, fewer adds for long 1-runs, ideal for signed arithmetic.
- **Best case**: long consecutive 1s (e.g., 01111110) $\rightarrow$ two operations.
- **Worst case**: alternating bits (010101010) $\rightarrow$ every step active.

Fast Multiplication and Carry-Save Adders

Motivation for Fast Multiplication

In binary multipliers, the total time depends on:

1. **Number of partial products, and**
2. **Time to add these partial products.**

Conventional **Shift-Add** or **Array multipliers** perform *serial additions*, one after another, leading to delays proportional to **n (bit width)**.

To accelerate this, **fast multiplication techniques** aim to:

- **Reduce the number of partial products, and**
- **Add multiple partial products in parallel.**

Two major speed-up techniques:

1. **Bit-Pair Recoding (reduces partial products)**
2. **Carry-Save Addition (speeds up addition of many partial products)**

Bit-Pair Recoding of Multipliers

Derived directly from Booth's algorithm.

Instead of examining one bit at a time, **we group multiplier bits in pairs.**

Idea:

Each pair of bits produces **at most one summand** of the multiplicand.

The **combination of bits determines the multiple of M to add ($0 \times M, \pm 1 \times M, \pm 2 \times M$).**

Examples of Recoding

Pair	Equivalent Operation
------	----------------------

(+1, -1)	(0, +1)
----------	---------

(+1, 0)	(0, +2)
---------	---------

(-1, +1)	(0, -1)
----------	---------

Thus, for ***n-bit multiplier***, only **$n / 2$ partial products** are generated.

This halves the number of additions required in the subsequent stages

Carry-Save Addition (CSA)

Concept

When many partial products are produced (for example in an Array or Wallace-tree multiplier), we must add them efficiently.

The **Carry-Save Adder** reduces the delay by **saving carries instead of propagating them immediately**.

Instead of waiting for carries to ripple through each column, the carry bits are **saved** and introduced into the next row — hence *carry-save*.

Structure

A CSA accepts **three n-bit inputs**:

- X, Y, Z
and produces **two n-bit outputs**:
- **Sum vector (S)** and
- **Carry vector (C)**

Each bit position i uses **one Full Adder (FA)**:

$$S_i = X_i \oplus Y_i \oplus Z_i$$

$$C_{i+1} = \text{majority}(X_i, Y_i, Z_i)$$

The **S and C vectors** are **not fully added**; they represent intermediate results to be combined later.

Timing Advantage

- Ripple-Carry Adder (RCA): delay $\approx n \times (\text{FA delay})$
- Carry-Save Adder: delay $\approx 1 \text{ FA delay}$, since all bit-positions operate **in parallel**.

Hence, partial products can be combined much faster.

Example

Adding three 4-bit numbers (A, B, C):

Bit A B C S C(out)

3 1 0 1 0 1

2 0 1 1 0 1

Bit A B C S C(out)

1 1 1 0 0 1

0 0 0 1 1 0

Output:

S = 0011, C = 1110 (left-shifted by one bit when re-added later).

Wallace Tree Multiplier

Purpose

To further speed up addition of multiple partial products using *hierarchical CSA stages*.

Principle

- Each stage reduces three input vectors into two using **3-2 reducers** (carry-save adders).
- Repeated until only **two vectors** remain.
- The final two vectors are added by a **fast adder** (like CLA).

Structure Overview

1. **Partial Product Generation:**
Using AND gates → produces $k = n$ partial products.
2. **Partial Product Reduction:**
 - Group every 3 rows → apply Carry-Save Adder (3-2 reducer).
 - Repeat until only 2 rows remain.
3. **Final Addition:**
 - Use **Carry-Lookahead Adder (CLA)** for final result.

3-2 Reducer

Each 3-2 reducer accepts three bits of the same weight and outputs:

- one **sum bit** (same weight),
- one **carry bit** (weight + 1).

Hence, it *reduces three operands into two* in one FA delay.

Example (6×6 Multiplication)

Six partial products: A – F

After 3 CSA levels (each ~ 1 FA delay), only two vectors remain (S_4, C_4).

Final CLA produces the product.

So, **Total delay ≈ 3 FA + CLA delay** — much less than serial addition.

Delay Estimation Formula

For k summands, the number of CSA levels needed $\approx$

$$\text{CSA} = 1.7 \log_2 k - 1.7$$

Example:

- For $k = 16 \rightarrow 1.7 \times 4 - 1.7 \approx 5.1$ levels
- For $k = 32 \rightarrow 1.7 \times 5 - 1.7 \approx 6.8$ levels

Practical Case: 32×32 Multiplier

- **8 CSA levels needed**
- Each FA ≈ 2 gate delays
- $8 \text{ levels} \times 2 = 16$ gate delays
- **Final CLA stage = 12 gate delays**
- Include 1 AND gate stage

$$\text{Total Delay} = 16 + 12 + 1 = 29 \text{ gate delays}$$

Fast Multiplier Delay = 29 gate delays

If RCA used instead of CLA:

$$6(n-1) - 1 = 185 \text{ gate delays}$$

Division section 6.6

Binary Division Algorithms – Restoring and Non-Restoring

Binary division in computer architecture is implemented using hardware algorithms designed to mimic longhand division. The two most widely used methods are the Restoring and Non-Restoring Division algorithms. Both operate on registers using repetitive shift and arithmetic operations to compute the quotient and remainder.

1. Restoring Division Algorithm

This method performs repeated subtraction of the divisor from the partial remainder, restoring the value whenever a subtraction leads to a negative result. It is mainly used for unsigned or positive signed binary numbers.

Steps:

- 1. Initialize registers: M (Divisor), Q (Dividend), $A = 0$ (n+1 bits).
- 2. Repeat n times:
 - Left shift (A,Q) by one bit.
 - Subtract M from A ($A = A - M$).
 - If $\text{sign}(A) = 1$, restore ($A = A + M$) and set $q_0 = 0$; else $q_0 = 1$.
- 3. After n iterations: A holds the remainder, Q holds the quotient.

Example: For Dividend = 7 (0111) and Divisor = 6 (0110), after 4 cycles $\rightarrow$ Quotient = 0010 (2), Remainder = 0001 (1).

Advantages: Simple to understand and implement in hardware. Disadvantages: Requires an additional restoration addition, increasing operation count.

2. Non-Restoring Division Algorithm

The non-restoring algorithm improves speed by eliminating the extra restoration step. Instead, it chooses the next arithmetic operation based on the sign of A. This makes it suitable for high-speed microprocessors and arithmetic logic units (ALUs).

Steps:

- 1. Initialize: M = Divisor, Q = Dividend, A = 0.
- 2. Repeat n times:
 - Left shift (A,Q) by 1 bit.
 - If $A \geq 0$, $A = A - M$ and $q_0 = 1$.
 - If $A < 0$, $A = A + M$ and $q_0 = 0$.
- 3. After n iterations, if $A < 0 \rightarrow A = A + M$ (final restoration).
- 4. A = Remainder, Q = Quotient.

Example: For Dividend = 11 (1011) and Divisor = 4 (0100), Quotient = 0010 (2), Remainder = 0010 (2).

Advantages: Faster, as it saves one addition per cycle. Disadvantages: Requires slightly more complex control logic for sign-based decisions.

3. Comparison Between the Two Methods

Feature	Restoring Division	Non-Restoring Division
Restoration Step	Performed after each negative result	Skipped; handled in next iteration
Speed	Slower	Faster (~50% fewer ALU operations)
Control Logic	Simpler	Slightly complex (sign-dependent)

Hardware Operations	$\approx 2n$	$\approx n$
Final Adjustment	Automatic	One-time correction if $A < 0$

4. Handling Signed Numbers

Both algorithms work for unsigned and positive signed operands. For signed division, magnitudes are used during division, and the sign of the result is determined separately.

- Rules:
 - $\text{Sign(Quotient)} = \text{Sign(Dividend)} \text{ XOR } \text{Sign(Divisor)}$
 - Remainder has the same sign as Dividend.
 - Example: $-7 \div 2 \rightarrow \text{Quotient} = -3, \text{Remainder} = -1$.

5. Performance Summary

Restoring Division performs approximately twice as many arithmetic operations as Non-Restoring Division. Hence, Non-Restoring Division is more suitable for high-performance hardware like CPUs, DSPs, and GPUs.

Both methods rely on sequential shifting and conditional arithmetic, with non-restoring being the foundation of modern divider circuits.

- Summary:
 - Restoring Division $\rightarrow$ simpler but slower.
 - Non-Restoring Division $\rightarrow$ faster, widely adopted in processors.

Floating point representation

Section 6.7.1

Introduction

Floating point representation is used to express real numbers that are too large or too small to be represented in fixed-point format.

It follows the scientific notation: $N = \pm M \times B^E$ where M (mantissa or significand) represents precision, B is the base (usually 2), and E is the exponent.

IEEE-754 standard defines Sign bit, Exponent bits, and Mantissa bits for representing real numbers.

Fixed-Point vs Floating-Point

Fixed point uses a fixed decimal location, giving limited range and precision.

Floating point uses variable position of the decimal, allowing much wider range.

Example:

Fixed-point: $0110.1100 = 6.75$

Floating-point: $1.101 \times 2^{-3} = 0.231$

IEEE-754 Floating Point Standard

IEEE-754 defines formats for single and double precision numbers.

Single precision (32 bits): 1-bit sign, 8-bit exponent, 23-bit mantissa, Bias = 127

Double precision (64 bits): 1-bit sign, 11-bit exponent, 52-bit mantissa, Bias = 1023

Formula: $N = (-1)^S \times (1 + M) \times 2^{(E - \text{Bias})}$

Biased Exponent Concept

Exponent is stored with a bias to represent both positive and negative exponents.

Single precision bias = 127 $\rightarrow$ Actual exponent = Stored - 127 (range: -126 to +127)

Double precision bias = 1023 $\rightarrow$ range: -1022 to +1023

Example: Exponent = $10000110_2 = 134 \rightarrow$ Actual exponent = $134 - 127 = 7$

Examples

(a) $0x43640000 \rightarrow$ Sign=0, Exponent=134 $\rightarrow E=7$, Mantissa= $1.110010_2 = 1.78125 \rightarrow 1.78125 \times 2^7 = 228$

(b) $0xBE200000 \rightarrow$ Sign=1, Exponent=124 $\rightarrow E=-3$, Mantissa= $1.010_2 = 1.25 \rightarrow -1.25 \times 2^{-3} = -0.15625$

(c) $-58.25_{10} \rightarrow 111010.01_2 = 1.1101001 \times 2^5$; Exponent= $127+5=132=10000100_2$;
Mantissa= $1101001...$
 $\rightarrow$ Hex = $0xC2690000$

Range of Values

Smallest normalized: $E=-126 \rightarrow \pm 1.0 \times 2^{-126} \approx \pm 1.175 \times 10^{-38}$

Largest normalized: $E=+127 \rightarrow \pm 2.0 \times 2^{127} \approx \pm 3.402 \times 10^{38}$

Special Cases

Zero: Exponent=0, Mantissa=0 $\rightarrow \pm 0$

Infinity: Exponent=all 1s, Mantissa=0 $\rightarrow \pm \infty$

NaN: Exponent=all 1s, Mantissa $\neq 0 \rightarrow$ Not a Number (e.g., $\sqrt{-1}$)

Denormalized: Exponent=0, Mantissa $\neq 0 \rightarrow$ very small numbers

Rounding Modes

IEEE defines four rounding modes:

1. Round down (toward $-\infty$)
2. Round up (toward $+\infty$)
3. Round toward zero
4. Round to nearest (default)

Example: Round 1.100101 (1.578125) to 3 fraction bits $\rightarrow$ Nearest = 1.101 (1.625)

= Typical Questions

- What is the significance of mantissa normalization? $\rightarrow$ Ensures unique representation.
- What happens when exponent=255 and mantissa $\neq 0$? $\rightarrow$ NaN.
- Decimal value of $0xC0000000$? $\rightarrow (-1)^1 \times 1.0 \times 2^1 = -2.0$

Summary

Floating point = Sign + Exponent + Mantissa.

Bias enables unsigned storage of signed exponents.

Precision defines accuracy; exponent defines range.

IEEE-754 ensures standardization, portability, and predictable rounding.

Basic Processing Unit- section 7.1

1. Register Transfer Concepts

Register transfer is a fundamental operation in digital computer organization, representing the movement of data between registers within a processor. Each register is constructed using edge-triggered D flip-flops and is controlled through specific control signals—Rin (Register Input Enable) and Rout (Register Output Enable). The transfer is synchronized with the clock signal to ensure precise timing.

- When Rout = 1 → the register places its contents onto the common internal bus.
- When Rin = 1 → the register loads data from the bus at the next clock edge.
- Each transfer operation is typically completed within one clock cycle.
- Control signals are generated by the instruction decoder during execution.

2. Implementation of a Register Bit

Each bit of a register (Ri) is implemented using a D flip-flop, a two-input multiplexer, and a tri-state gate. The multiplexer selects between the data on the bus and the current stored value in the flip-flop. The control input Ri_in determines which source is selected. When Ri_in = 1, the flip-flop captures bus data at the rising clock edge. When Ri_in = 0, the flip-flop retains its previous state. The output Q is connected to the system bus through a tri-state gate controlled by Ri_out. When Ri_out = 0, the gate is in a high-impedance state, effectively isolating the register from the bus. When Ri_out = 1, the stored value is driven onto the bus.

3. Basic Processing Unit Components

The processor's Basic Processing Unit (BPU) includes several interconnected components:

- Arithmetic and Logic Unit (ALU): Executes arithmetic and logic operations.
- Processor Registers: General-purpose and special-purpose registers for temporary data storage.
- Internal Bus: Facilitates data transfer among the ALU, registers, and memory interface.
- Memory Address Register (MAR): Holds the address of the memory location to access.
- Memory Data Register (MDR): Temporarily stores data being transferred to/from memory.
- Control Unit: Generates timing and control signals for synchronization.

4. Instruction Execution Phases

Instruction execution occurs in multiple phases:

- **Fetch Phase:** Retrieves the instruction from memory and loads it into the Instruction Register (IR).
- **Decode Phase:** The instruction decoder interprets the operation and generates control signals.
- **Execute Phase:** Performs the specified operation—data transfer, ALU computation, or memory access.

Each phase may span one or more clock cycles depending on instruction complexity.

5. Memory Access Operations

Two key memory operations are performed:

- **Fetching (Read):** MAR holds the address; the processor sends a Read signal. MDR receives data from memory.
- **Storing (Write):** MAR holds the address, MDR holds the data, and a Write signal initiates the transfer.

During these operations, the Memory Function Completed (MFC) signal indicates completion. Cache memory responds in one cycle, whereas main memory may take multiple cycles.

6. Example: Move (R1), R2 Instruction

Example sequence for Move (R1), R2:

1. R1out, MARin — Transfer memory address from R1 to MAR.
2. Read, MDRinE — Initiate memory read and wait for MFC.
3. MDRout, R2in — Transfer data from MDR to R2.

This example illustrates how register control signals, bus operations, and memory interface work together.

The Basic Processing Unit, with its registers, buses, and ALU, forms the heart of the processor datapath. Understanding register transfer and control timing is essential for designing efficient digital systems.

Execution of a Complete Instruction –

Execution in a processor involves a series of coordinated micro-operations controlled by the control unit. Each instruction is fetched from memory, decoded to determine the operation, and then executed through the processor's datapath components such as registers, ALU, buses, and memory interface units. In a **single-cycle datapath**, these steps are organized in such a way that each operation (fetch, decode, execute) is completed in one or more well-defined control steps synchronized with the system clock.

1. Instruction Fetch and Execution Flow

The execution of a complete instruction typically begins with the **instruction fetch** phase. The contents of the Program Counter (PC) are sent to the **Memory Address Register (MAR)** via the ALU or through a direct path. A **Read** command is issued to memory, and simultaneously, the PC is incremented by 4 to point to the next instruction. This incrementing can be done either by the main ALU or, in more optimized architectures, by a dedicated **Incrementer unit**. The data fetched from memory is stored in the **Memory Data Register (MDR)** and then transferred to the **Instruction Register (IR)**. The instruction is then decoded by the control logic to identify the type of operation (arithmetic, logical, memory, or control).

In the **execution phase**, the operands are fetched, the specified operation is performed, and the result is stored in the appropriate register or memory location. For example, in an instruction like **Add (R3), R1**, the address in register R3 is used to fetch a memory operand, which is then added to the contents of R1, and the result is stored back in R1. This process involves micro-operations such as **R3out, MARin, Read**, followed by waiting for the **Memory Function Complete (MFC)** signal, and then performing the ALU operation and writing the result.

Execution of Branch Instructions

A **branch instruction** alters the normal sequential flow of execution by replacing the contents of the PC with a new target address. The branch target address is obtained by adding an **offset X** to the address immediately following the branch instruction. For example, if a branch instruction is located at address 2000 and the branch target address is 2050, the offset X equals 46 (since $2050 - 2004 = 46$).

Unconditional branch instructions always load the branch target into the PC. However, **conditional branch instructions** perform this operation only when certain condition flags are satisfied. For instance, a “Branch-on-Negative” instruction checks the N (Negative) flag before updating the PC. The control sequence includes an additional decision step such as: **Offset-field-of-IRout, Add, Zin, If N = 0 then End**. If $N = 1$, the PC is updated with the new address ($Zout \rightarrow PCin$), completing the branch. Otherwise, execution proceeds sequentially. This mechanism allows efficient implementation of decision-making and loop control at the hardware level.

Role of the Incrementer Unit

The **Incrementer** is a small hardware unit dedicated to incrementing the PC by a constant value (usually 4, corresponding to the instruction word size in a byte-addressable memory). Earlier, this operation was performed by the ALU, which added extra load and delay to the datapath. By using a separate Incrementer, the architecture avoids ALU involvement for PC updates, thereby streamlining instruction fetch operations.

Even though the Incrementer replaces the ALU for PC updates, the **constant 4** remains available as a selectable input at the ALU multiplexer. This constant is still useful for incrementing other addresses, such as those used in **LoadMultiple** or **StoreMultiple** instructions, which access consecutive memory locations in block data transfers.

In summary, the Incrementer enhances performance by enabling parallelism: the ALU can perform computation while the Incrementer prepares the address of the next instruction.

Multiple Bus Organization

To further improve data transfer efficiency, many architectures adopt a **multiple bus organization**. A conventional single-bus structure allows only one data transfer per clock cycle, which limits throughput. In contrast, a multiple-bus design (typically a three-bus system) enables simultaneous operations.

The **register file** in such a design has three ports: two **output ports** and one **input port**. The two output ports drive **buses A and B**, which supply operands to the ALU, while the input port receives data from **bus C**, which carries the result back to a destination register. This allows instructions like Add R4, R5, R6 to execute in one cycle: R5 and R6 are read simultaneously via buses A and B, processed by the ALU, and the result is written to R4 through bus C.

Additionally, the ALU can output one of its input operands directly to bus C without modification (e.g., when controlled by signals such as $R=A$ or $R=B$). This flexibility supports both arithmetic and data movement instructions efficiently. The multiple bus organization therefore minimizes bottlenecks in operand fetching and result storage, leading to faster instruction throughput.

Control Sequence Example

The control sequence for executing an instruction such as Add (R3), R1 illustrates the integration of all the above components:

1. **Instruction Fetch:** $PC_{out} \rightarrow MAR_{in}$, Read, $PC \leftarrow PC + 4$.
2. **Wait for MFC:** Data received into MDR.
3. **Instruction Load:** $MDR_{out} \rightarrow IR_{in}$.
4. **Operand Fetch:** $R3_{out} \rightarrow MAR_{in}$, Read, WMFC, $MDR_{out} \rightarrow Y_{in}$.
5. **Execution:** $R1_{out}$, Add, Z_{in} ; $Z_{out} \rightarrow R1_{in}$.

Each step corresponds to a distinct clock cycle with well-defined control signals, ensuring proper synchronization of data transfers and ALU operations.

The execution of complete instructions in a processor depends on efficient coordination between the **control unit**, **datapath elements**, and **memory interface**. The use of an Incrementer for PC updates, conditional control sequences for branching, and multiple bus structures for parallel data movement together contribute to a high-speed execution cycle. These architectural optimizations ensure that instruction throughput is maximized without compromising control precision, forming the foundation for modern pipelined and superscalar processors.

Hardwired and Microprogrammed Control

1. Introduction

The control unit of a computer is responsible for generating control signals that coordinate all internal operations of the processor. These signals determine the sequence of micro-operations required to execute each instruction. Computer designers have developed two major methods for designing control units — **Hardwired Control** and **Microprogrammed Control**. While hardwired control uses fixed logic circuits to directly produce control signals, microprogrammed control uses a small control memory that stores a program of microinstructions to generate these signals sequentially. Both approaches achieve the same goal but differ in speed, flexibility, and complexity.

Hardwired Control

A **hardwired control unit** is implemented using combinational and sequential logic elements such as decoders, flip-flops, and encoders. Each control signal is generated directly by hardware components, which respond to the current instruction, clock cycle (or control step), and condition flags.

The **control step counter** keeps track of the current phase of instruction execution. For each clock pulse, the **RUN** signal increments the counter by one, progressing through micro-operations. When **RUN = 0**, the counter halts—usually when the processor issues a **WMFC (Wait for Memory Function Completed)** signal and waits for memory to respond.

Another important signal, **END**, resets the control step counter, marking the completion of an instruction cycle and allowing the next instruction fetch to begin. These control signals are combined using encoder-decoder logic blocks that generate appropriate outputs such as Yin, PCout, Add, and End.

Each control signal is produced according to a particular step and instruction. For instance, the control signal Zin may be active in T1 for all instructions and in T6 only for the ADD instruction. Thus, the hardwired control operates as a **state machine**, where each state corresponds to one control step.

Advantages:

Hardwired control provides **very high speed** since there is no memory access delay for control signals. It is suitable for **RISC architectures** that use simple and uniform instruction sets.

Disadvantages:

It is **less flexible**, as changes in instruction design require modifying hardware logic. It also becomes increasingly **complex** for large instruction sets.

A Complete Processor

A typical processor includes several interconnected units: an **instruction unit**, **integer unit**, **floating-point unit**, **data cache**, and **bus interface**.

- The **instruction unit** fetches instructions from cache or main memory.
- The **data cache** provides fast data access and stores frequently used values.

- The **bus interface** connects the CPU to the system bus, facilitating communication with I/O devices and main memory.

Modern processors may have **multiple integer and floating-point units** to enable **parallel execution** of operations. Separate caches for instructions and data are common, though some systems use a unified cache. These structures ensure faster processing and efficient data flow across different execution units.

Microprogrammed Control

In contrast to hardwired logic, **microprogrammed control** generates control signals using a **program stored in control memory (ROM or EEPROM)**. Each instruction's execution sequence is represented as a series of **microinstructions**, and each microinstruction contains a **control word (CW)** — a binary pattern where each bit activates a specific control signal (for example, PCout, Yin, Add, Zin).

The **microprogram counter (uPC)** keeps track of the current microinstruction address. It increments sequentially after each step unless a branch or end microinstruction is executed. For conditional operations, **branch microinstructions** are used, allowing the control unit to check condition flags or instruction bits and then jump to a new microinstruction sequence if needed.

For example, the instruction **Branch < 0** can be executed by testing the **Negative (N)** flag. If $N = 0$, control returns to the instruction fetch cycle; otherwise, the branch address is loaded into the Program Counter (PC).

Advantages:

Microprogrammed control is **flexible and easy to modify**, as new instructions can be added by updating the control memory instead of redesigning hardware. It simplifies the design of **complex instruction sets** (CISC).

Disadvantages:

It is **slower** than hardwired control because microinstructions must be fetched from control memory before execution.

Horizontal vs. Vertical Microprogramming

The organization of microinstructions determines the efficiency of a microprogrammed system:

- **Horizontal Microprogramming:**
 - Each microinstruction directly specifies many control signals (minimal encoding).
 - Enables parallel execution of multiple micro-operations.
 - Faster but requires longer control words and more memory.
 - Example: 20-bit microinstruction controlling 42 signals.
- **Vertical Microprogramming:**
 - Control signals are grouped and encoded, using fewer bits per microinstruction.

- Slower because signals are decoded sequentially.
- Saves memory space but increases control complexity.

Trade-off: Horizontal organization prioritizes speed and parallelism, while vertical organization prioritizes compactness and efficient memory use.

Comparison Between Hardwired and Microprogrammed Control

Aspect	Hardwired Control	Microprogrammed Control
Implementation	Logic circuits (decoders, counters)	Control memory storing microinstructions
Speed	Very fast	Slower due to memory fetch
Flexibility	Difficult to modify	Easily modified or updated
Complexity	Increases for large instruction sets	Simplifies complex instruction design
Usage	RISC processors	CISC processors

Summary

Hardwired control units achieve speed by generating control signals directly through logic, while microprogrammed control achieves flexibility by storing control sequences in memory. The **RUN** and **END** signals manage the sequencing of steps in hardwired designs, whereas the **microprogram counter (uPC)** performs the same role in microprogrammed systems.

Modern processors often adopt a **hybrid approach**, using hardwired logic for frequently executed simple operations and microprogrammed control for complex instructions. This combination ensures both performance efficiency and design flexibility, aligning with the needs of today's high-performance architectures.